Turret Storm

1. Project Overview

Turret Storm is a Tower Defense game built using Python and Pygame. Players strategically place defensive towers on a grid-based map to prevent waves of enemies from reaching their base. The game features multiple tower types (like arrow, cannon or ice), various enemy types with different attributes, an in-game economy system, and a statistics dashboard for gameplay analysis.

The game combines real-time strategy mechanics (RTS) with resource management, requiring players to balance their gold spending between placing new towers and upgrading existing ones. A built-in statistics system collects over 7 measurable features from gameplay, storing all data in CSV format for analysis and visualization.

2. Project Review

This project is inspired by classic tower defense games such as Bloons TD. Which feature wave-based enemy spawning, multiple tower types with upgrade paths, and strategic placement mechanics.

Key improvements and unique aspects of this project include:

- Built-in statistics collection system tracking with real-time visualization dashboard
- CSV data export for external analysis (like with Excel, pandas)
- Object-Oriented design with 6 classes
- Ice tower slow mechanic adding strategic depth
- Splash damage system for Cannon towers

3. Programming Development

3.1 Game Concept

Tower Defense is a real-time strategy game where the player defends a base by placing towers along a predetermined enemy path. The game is divided into waves, with each wave sending progressively more numerous and harder enemies.

Core Mechanics:

- Grid-based tower placement on grass tiles adjacent to the enemy path

- Three tower types: Arrow (fast and cheap), Cannon (slow with splash damage), Ice (slows enemies)
- Tower upgrades up to Level 3, increasing damage and range
- Four enemy types: Grunt (normal), Scout (fast), Tank (heavy), Boss (very tough)
- Gold economy: earn gold from kills and wave bonuses, spend on towers and upgrades
- Lives system: enemies reaching the end cost lives. game ends at 0 lives

3.2 Object-Oriented Programming Implementation

The system is modelled with six classes. Each class owns a single, clearly-scoped responsibility, which keeps the main update/draw loop small and makes the UML relationships (composition/aggregation / association) meaningful rather than decorative.

Game (game.py) - top-level controller. Holds the game state (playing/wave_active/game_over/victory), the gold and lives counters, and the lists of live towers and enemies. Drives the main update/draw cycle and dispatches keyboard and mouse input. Implemented as a single controller so that the update order (enemies → towers → projectiles → UI) lives in exactly one place.

GameMap (game_map.py) - tile grid and path. Stores the 12×10 grid from settings. GAME_MAP, knows which tiles are paths and which are grass, answers is_buildable(col, row) questions for the placement UI, and draws the map using sprites from assets.TILES. Separated from Game so that map data and tower-placement rules can be reasoned about (and later swapped for new maps) without touching game-state logic.

Tower (tower.py) - defensive structure. Tracks its type, level (1–3), gold invested, stats (damage / range / fire rate), live projectiles, selection state, and the rotation angle of its head. Each frame it picks the nearest enemy in range, rotates toward it, and fires when its timer expires. Implemented as its own class so that Arrow, Cannon, and Ice variants share one targeting/firing pipeline and differ only in stats and projectile behaviour.

Projectile (tower.py) Spawned by a Tower, moves toward its target, applies impact (single-target, splash, or slow), and reports hit/kill events back to its

owning tower so per-tower statistics stay accurate. Kept as a separate class because projectiles have their own update/collision lifecycle that is independent of the tower that fired them.

Enemy (enemy.py) - path-following unit. Walks between waypoints, accepts damage, can be slowed by Ice-tower projectiles (with a visible cold aura), and signals when it dies or reaches the end of the path. Implemented as a class so that Grunt/Scout/Tank/Boss variants share one movement-along-path pipeline and differ only in HP, speed, and reward.

StatsManager (stats_manager.py) - data pipeline. Exposes seven record_* methods the rest of the game calls when interesting events occur, renders an in-game dashboard cycled with TAB, and writes seven CSV files to stats_data/ at wave boundaries and on shutdown. Isolated in its own class so that gameplay code never performs file I/O and statistics can be extended without touching the game loop.

Class	Attributes	Methods
Game	state, wave_num, gold, lives, towers, enemies	update(), handle_key(), handle_click(), _start_wave(), _place_tower(), _upgrade_tower()
GameMap	grid, path_tiles, rows, cols	is_buildable(), pixel_to_grid(), grid_to_pixel_center(), draw()
Tower	tower_type, level, damage, range, kills, projectiles	find_target(), update(), upgrade(), _shoot(), get_sell_price(), draw()
Projectile	x, y, target, damage, speed, splash_radius	update(), draw()
Enemy	health, speed, position, waypoint_index, slow_timer	take_damage(), apply_slow(), update(), draw()
StatsManager	wave_stats, enemy_kills, gold_transactions, damage_events	record_*(), save_to_csv(), draw_dashboard(), toggle_dashboard()

3.3 Algorithms Involved

The game incorporates several key algorithms:

- Pathfinding: Enemies follow pre-defined waypoints along the path using linear interpolation between grid positions.

- Target Acquisition: Towers use nearest-enemy selection within range, computed via Euclidean distance.
- Projectile Physics: Projectiles track moving targets using velocity-based interpolation toward the target position.
- Splash Damage: Cannon projectiles check all enemies within a splash radius upon impact using distance calculations.
- Wave Scaling: Enemy count and health scale progressively based on wave number, with boss spawns every 5 waves.
- Slow Effect: Ice towers apply a time-based debuff that reduces enemy speed by a configurable factor.

4. Statistical Data (Prop Stats)

4.1 Data Features

The game tracks 7 measurable features:

No.	Feature	Unit of Record	Expected Records per Playtest
1	Wave Stats	one row per completed wave	10–15 (one per wave)
2	Tower Placements	one row per tower built	5–20 (placement events)
3	Enemy Kills	one row per lethal hit	150–400 (kills across waves)
4	Gold Transactions	one row per income or expense event	40–120 (builds, kills, bonuses)
5	Damage Events	one row per projectile hit	400–1,500 (tower-wave combos)
6	Wave Completion	one row per completed wave	10–15 (one per wave)
7	Player Actions	one row per click / keypress	100–500

A full victory run reaches wave 20, the counts above cover both a typical 10-wave playtest and a longer run. Across all seven tables a normal playtest writes on the order of a few thousand rows, which is well within what a single CSV can hold.

4.2 Data Recording Method

Data is collected during play by calling StatsManager.record_* methods from the rest of the codebase at the moment each event occurs:

- Game.place_tower() calls record_tower_placed() and record_gold_transaction().
- Tower.update() calls record_damage_event() on every hit and record_enemy_kill() when the hit is lethal.
- Game.start_next_wave() and the wave-end logic call record_wave_complete() and record_gold_transaction() for the wave bonus.
- Game.handle_key() / handle_click() call record_player_action().

Each record is a plain Python dict. StatsManager keeps them in per-feature lists in memory during a wave — so the hot game loop never performs file I/O — and calls save_to_csv() at two points:

1. At the end of every wave, a completed wave is already on disk before the next wave begins.
2. On shutdown (ESC or window close), any partial-wave records are also persisted.

Writing once per wave (rather than once per frame or once per event) keeps the per-frame overhead at zero and bounds the blast radius of a mid-game crash to at most the wave currently in progress. The output is seven CSV files in stats_data/, one per feature. The offline script data_analysis.py then reads those CSVs back with pandas and produces PNG charts plus a wave-summary table in stats_data/analysis/

4.3 Data Analysis Report

The game includes a built-in statistics dashboard (press TAB) with 3 pages of visualizations:

1. Overview Page: Total kills, damage, gold earned/spent, towers placed, game time, and record counts per feature.
2. Wave Charts: Bar charts showing enemies killed per wave, wave completion time, and gold earned per wave.
3. Tower Analysis: Pie chart of damage distribution by tower type, kill counts by tower and enemy type with horizontal bar charts.

For the final report, additional external analysis will be performed using Python (pandas + matplotlib) on the exported CSV data, including: descriptive statistics (mean, med, SD) for each feature, correlation analysis between tower placement strategy and wave performance, time-series analysis of player improvement across waves, and comparative analysis of tower type effectiveness.

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Tile map + fixed enemy path (GameMap), settings. GAME_MAP constants, basic map rendering
3	10	Enemy movement along path (Enemy), first wave spawner, lives counter
4	11	Tower placement UI (1/2/3 keys + left-click), Tower class, tower selection, gold/economy scaffolding
5	12	In-game TAB dashboard, right-panel UI polish, text-wrapping for tower descriptions, HUD (gold / lives / wave / FPS)
6	13	Bug-fix sprint: edge-case collision at low FPS, CSV flush on unexpected close, input-handling edge cases
7	14	Submission week (Draft)

6. Document version

Version: 1.5
Date: 12 March 2025
Editor: Navin Bunthuphanich 6610545251

Date	Name	Feedback
15/03	Peraya	**Overall Feedback:** While your game concept is strong and your event-based data logging is great, your proposal is incomplete. You have left an entire section undone, missing descriptions in your OOP architecture, and have a mathematical oversight in your data volume estimations. Major Changes Needed

		- **5. Project Planning Timeline (Incomplete Document)**
		  - Weeks 2 through 6 are entirely blank. A project proposal cannot be approved without a development roadmap.
		  - You must fill in the missing weeks with specific development milestones.
		- **3.2 OOP Implementation (Missing Class Roles)**
		  - You provided a table with Attributes and Methods which is good, but you completely omitted the **Role** or purpose of each class. Why do these classes need to exist? You must add a description for each of the 6 classes explaining what they actually do in the game.
		
		Minor Recommendations & Changes
		- **4.2 Data Recording Method (Memory Management)**
		  ○ Storing all data in memory (lists/dicts) until the game closes is risky for high-frequency data like "Player Actions" (tracking every single click). If the game crashes, all data is lost.
		  ○ It is safer to write/append to the CSV file periodically rather than holding thousands of rows in RAM.
		- **Proofreading & Formatting:** Don't forget to do a final proofreading before the final proposal.

20/03	Amornrit	Solid game design. However, your proposal is incomplete; each proposed class requires an explanation, and you should have planned your project by now. One thing to watch is that real-time logging of every click, keypress, kill, and gold change can grow quickly and add overhead if the **StatsManager** does too much work during gameplay. A good improvement is to keep logging lightweight by storing only essential fields in memory, aggregating high-frequency data where possible, and exporting in batches at wave end or on shutdown. It is also worth adding an occasional autosave or periodic flush, since relying only on game close could lose data if the game crashes unexpectedly.